

Lecture 17: Page Replacement Algorithms

Professor G. Sandoval

17



Some slides adapted by G. Sandoval for CS3224,
from slide by Brendan Dolan-Gavitt

```

    if (groupinfo->blocks[0] != group_info->small_
        int i;
        if (groupinfo->blocks[0] != group_info->small_
            for (i = 0; i < group_info->nblocks; i++)
                int i;
                freepage((unsigned long)groupinfo->blo
                for (i = 0; i < group_info->nblocks; i++)
                    }
                    freepage((unsigned long)groupinfo->blo
                    kfree(groupinfo);
                    }
                    kfree(groupinfo);
                }
            }
        }
        EXPORT_SYMBOL(groupsfree);
        EXPORT_SYMBOL(groupsfree);
        /* export the groupinfo to a user-space array */
        /* export the groupinfo to a user-space array */
        const struct group_info *group_info)
        static int groups_touser(gid_t_user *grouplist,
        {
            const struct group_info *group_info)
            int i;
            unsigned int count = groupinfo->ngroups;
            int i;
            unsigned int count = groupinfo->nblocks;
            for (i = 0; i < group_info->nblocks; i++) {
                unsigned int cpcount = min(NGROUPSPERBLOCK
                for (i = 0; i < group_info->nblocks; i++) {
                    unsigned int len = cpcount * sizeof(*group
                    unsigned int cpcount = min(NGROUPSPERBLOCK
                    = cpcount * sizeof(*group

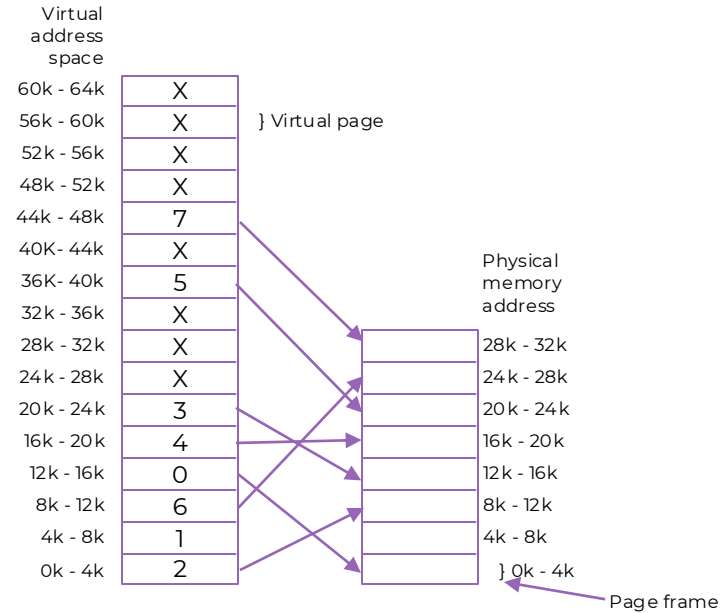
```

Page Replacement Algorithms XV6 Implementation

Page Replacement Algorithms

- If we **can't fit everything in memory**, eventually we will have to choose something to **evict** and write to disk
- As with scheduling algorithms, this is a well-studied area and there are lots of strategies
- This general principle shows up any time we have a **limited-size cache** – strategies described here apply to all of them

Paging (reminder)



The relation between virtual addresses and physical memory addresses is given by the page table. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K–8K really means 4096–8191 and 8K to 12K means 8192–12287

Cache Management

T_m = time to access memory

T_d = time to access disk

P_{miss} = Prob of not finding the data in cache

Remember:

Nano = 10^{-9}

Micro = 10^{-6}

Mili = 10^{-3}



- Average Memory Access Time (AMAT)

$$AMAT = T_m + (P_{miss} * T_d)$$

- Assume:
 - $T_m = 100$ nanoseconds
 - $T_d = 10$ milliseconds

Class 17: A memory access takes 100 ns if it hits in RAM. A page fault takes 10 ms (disk). If 1% of memory accesses cause a page fault, the average memory access time is closest to:

10^2 ns (≈ 100 ns)

0%

10^3 ns ($\approx 1,000$ ns)

0%

10^5 ns ($\approx 100,000$ ns)

0%

10^7 ns ($\approx 10,000,000$ ns)

0%

AMAT calculations

- If $P_{\text{miss}} = 0.1$, what's the AMAT?
 - $100\text{ns} + 0.1 * 10\text{ms} = 1.0001\text{ ms!!!}$
- What if $P_{\text{miss}} = .001$?
 - $100\text{ns} + 0.001 * 10\text{ms} = 10\mu\text{s} \text{ !! } 100 \text{ times faster}$

The "Optimal" Algorithm

- At the time of a fault, consider the set of pages in memory
- Given the code executing, they will each be referenced some number of instructions from now
- To choose one to evict, just **pick the one that's furthest from being referenced**

What is the main goal of a page replacement algorithm?

Decide which process to run next on the CPU

0%

Decide which page to evict from RAM on a page fault

0%

Decide the size of each page

0%

Decide how big virtual memory should be

0%

The "Optimal" Algorithm

"Prediction is very difficult, especially about the future."

– Niels Bohr

- Problem: requires the OS to **predict the future**
- Still, it's useful as a goal to work toward
- We can **benchmark other algorithms** by how close they get to being optimal

Tracing the Optimal Policy

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

$$\begin{aligned}\text{Hit Rate} &= \text{Hits} / (\text{Hits} + \text{Misses}) \\ &= 6 / (6+5) = 54.5.\end{aligned}$$

The Optimal page replacement algorithm evicts the page that:

Has not been used for the longest time in the past

0%

Will not be used for the longest time in the future

0%

Has the smallest page number

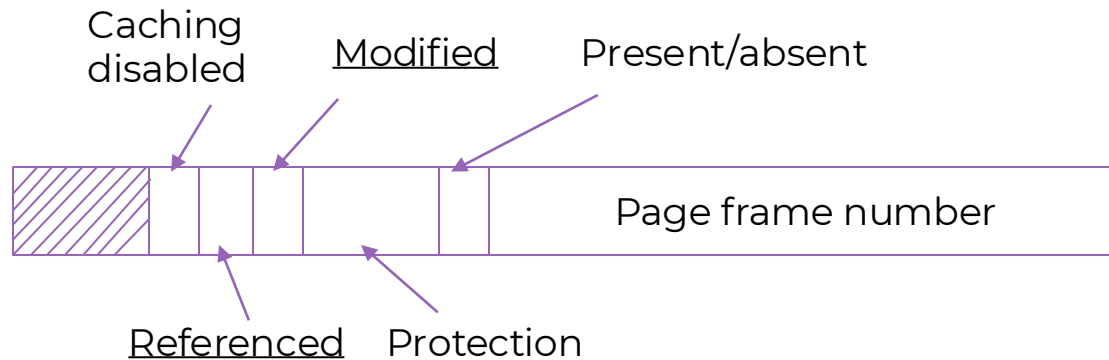
0%

Has never been modified (clean)

0%

Page Bits

- Remember that page table entries contain bits that indicate whether the page was recently *referenced* or *modified* (R & M)



typical page table entry.

Not Recently Used

- We can use these bits to track which pages in memory have not been touched in a while
- If a page hasn't been used in a while, it may be a good candidate for eviction

Implementing Not Recently Used

- If we just rely on the CPU to mark referenced pages, over time eventually all pages would be referenced
- Instead we can have the OS periodically clear the referenced bit (say, every clock tick)
- Now if the referenced bit is 0, we know the page has not been referenced in at least one clock tick

Implementation

- When a page fault occurs, the OS inspects all the pages and divides them into 4 categories based on values of R & M bits:
- **Class 0:** not referenced, not modified
- **Class 1:** not referenced, modified
- **Class 2:** referenced, not modified
- **Class 3:** referenced, modified
- Clock interrupts clear the R bit but NOT the M bit.
- NRU **removes** a page at random from the lowest numbered nonempty class.

Class 17: NRU divides pages into 4 classes using (R, M). Which class is best to evict first?

(R=1, M=1) – referenced, modified

0%

(R=1, M=0) – referenced, clean

0%

(R=0, M=1) – not referenced, modified

0%

(R=0, M=0) – not referenced, clean

0%

First In, First Out (FIFO)

- Simple algorithm:
 - Keep a linked list of pages in the order they were brought into memory
 - Tail is most recent, head is oldest
 - To evict, throw out the one at the head of the list
- **But:** just because it's the oldest doesn't mean it's not being used!

Tracing the FIFO Policy

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss ✓		First-in→ 0
1	Miss ✓		First-in→ 0, 1
2	Miss ✓		First-in→ <u>0, 1, 2</u>
0	Hit ✓		First-in→ 0, 1, 2
1	Hit ✓		First-in→ 0 , 1, 2
<u>3</u>	<u>Miss</u>	<u>0</u>	<u>First-in→ 1, 2, 3</u>
<u>0</u> ✓	<u>Miss</u>	<u>1</u>	<u>First-in→ 2, 3, 0</u>
3	Hit		First-in→ 2, 3, 0
1	Miss	2	First-in→ 3, 0, 1
2	Miss	3	First-in→ 0, 1, 2
1	Hit		First-in→ 0, 1, 2

$$\text{Hit Rate} = \text{Hits} / (\text{Hits} + \text{Misses})$$

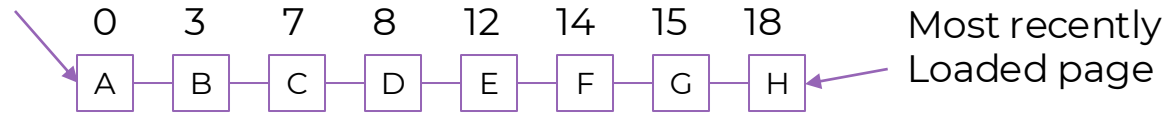
$$= 4 / (4 + 7) = 36 \%$$

Second Chance

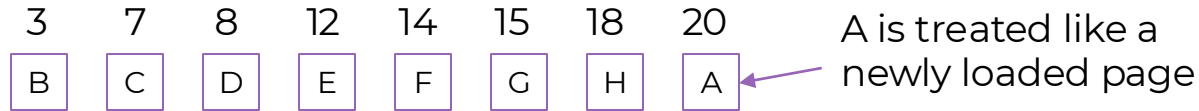
- Since we have the referenced bit available, we can do slightly better than FIFO
- Before throwing out the oldest page, check if it's been **referenced**
- If it has, clear the referenced bit and move it to the back of the list (as though it were a new page)
- Keep looking through the list for something to evict

Second Chance

Page loaded first



(a)



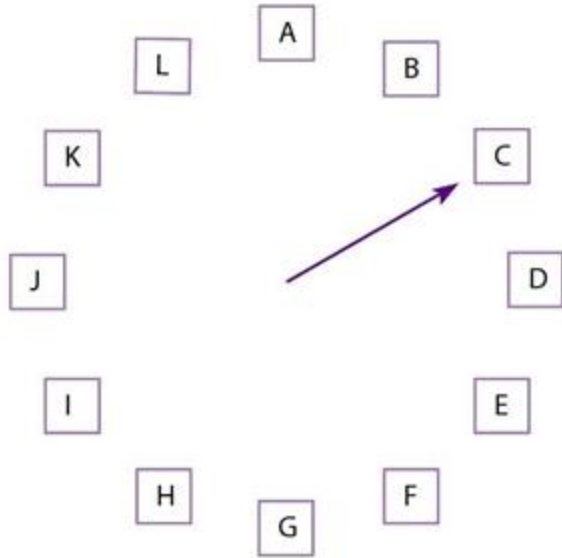
(b)

Figure 3-15. Operation of second chance. (a) Pages sorted in FIFO order. (b) Page list if a page fault occurs at time 20 and A has its R bit set. The numbers above the pages are their load times.

Clock

- FIFO/Second Chance is inefficient because it may have to move around entries in the list often
- As an optimization, we can imagine the pages arranged in a circle and keep a pointer (or index) indicating the position of the "clock hand"
- Clock hand points to the oldest page – so now we can apply Second Chance by just updating the position of the clock hand

Clock



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

R = 0: Evict the page

R = 1: Clear R and advance hand

Figure 3-16. The clock page replacement algorithm.

Least Recently Used (LRU)

- **Observation:** pages that have been used a lot recently will probably be used again soon
- So a good strategy might be: throw out the least *recently used* page

Tracing the LRU Policy

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		LRU→ 0
1	Miss		LRU→ 0, 1
2	Miss		LRU→ 0, 1, 2
0	Hit		LRU→ 1, 2, 0
1	Hit		LRU→ 2, 0, 1
3	Miss	2	LRU→ 0, 1, 3
0	Hit		LRU→ 1, 3, 0
3	Hit		LRU→ 1, 0, 3
1	Hit		LRU→ 0, 3, 1
2	Miss	0	LRU→ 3, 1, 2
1	Hit		LRU→ 3, 2, 1

$$\text{Hit Rate} = \text{Hits} / (\text{Hits} + \text{Misses}) \\ = 6 / (6+5) = 54.5\%$$

As good as optimal!!

LRU Implementation

- Expensive to Implement!! 4GB of mem with 4KB pages. 1MM page lookup!
- One possibility: keep a list of pages (as with FIFO) and update it every time memory is referenced
- Or, if we have extra hardware support – have hardware write a timestamp to the PTE every time every time a page is referenced
- Neither of these is particularly appealing

Approximating LRU: Not Frequently Used (NFI)

- We can (approximately) simulate LRU in software reasonably cheaply though
- Maintain a list of counters, one per page
- At each clock tick, if a page has been referenced, update its counter
- Now we have a rough count of how often each page has been referenced over time

NFU Problems

- A page might be referenced very often early on, and then never referenced again
- But its count will remain high for a long time, preventing it from getting evicted
- Meanwhile, a page that is referenced periodically (say every 20 ticks) may have a lower count but be in active use

Aging

- To solve this problem, we can have counter values decay over time
- Each clock tick, shift all counters right by one bit
- Add in the reference bit as the leftmost bit
- To **evict**, just **choose the lowest counter value** –because more recent references are in the more significant digits, they have greater weight

Class 17: What does the "R" (Referenced) bit in a page table entry tell the OS?

A) Whether the page is currently in RAM

0%

Whether the page has been modified (written to)

0%

Whether the page has been accessed recently

0%

Whether the page belongs to the kernel

0%

Aging

	R bits for page 0-5, Clock tick 0	R bits for page 0-5, Clock tick 1	R bits for page 0-5, Clock tick 2	R bits for page 0-5, Clock tick 3	R bits for page 0-5, Clock tick 4
	1 0 1 0 1 1	1 1 0 0 1 1	1 1 0 1 0 1	1 0 0 0 1 0	0 1 1 0 0 0
Page					
0	10000000	11000000	11100000	11110000	01111000
1	00000000	10000000	11000000	01100000	10110000
2	10000000	01000000	00100000	00010000	10001000
3	00000000	00000000	10000000	01000000	00100000
4	10000000	11000000	01100000	10110000	01011000
5	10000000	01000000	10100000	01010000	00101000
	(a)	(b)	(c)	(d)	(e)

LRU Approximation

- Aging is only an approximation of an actual LRU algorithm:
 - We don't get any information about how often something was referenced between two clock ticks
 - If our counter is N bits, our memory only extends back N clock ticks – so we can't distinguish how old things are past that point

Working Set

- Most processes don't reference pages randomly scattered around memory
- Instead, they have *locality of reference* – tend to reference only a small set of pages in a given time period
- We call the set of pages currently being used by a process its *working set*
- If you don't have enough memory to hold the working set, you will constantly be swapping – known as *thrashing*

Prepaging

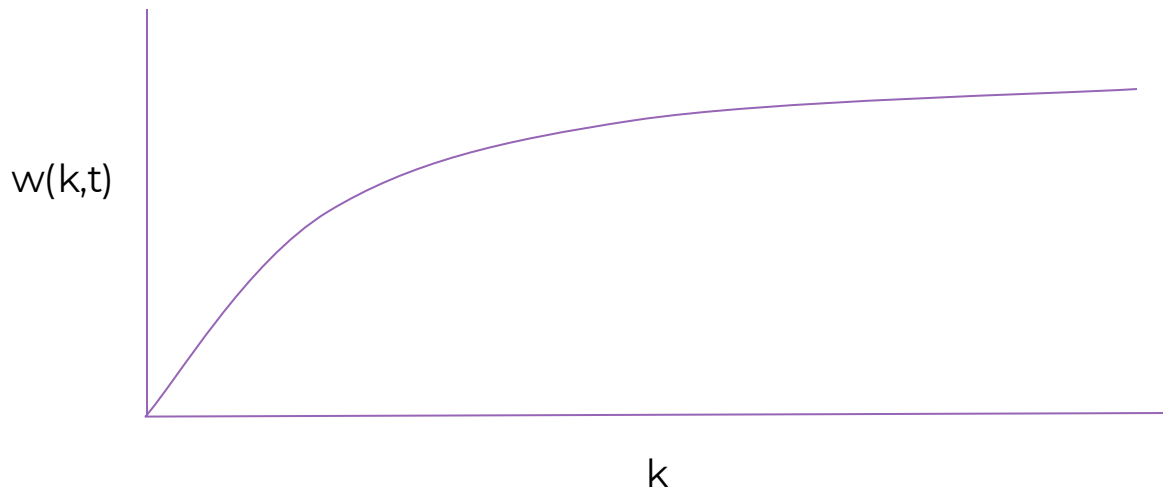
- When we want to bring a process back in after swapping it in, we have the option of doing nothing
 - When the process runs it will just continually page fault until all pages in the working set are in memory
- Downside: each page fault requires some time to process – this is *slow*
- Instead we can try to load the process's working set all at once; this is called *prepaging*

Size of the Working Set

- The working set $w(k,t)$ is more precisely defined as the number of pages referenced in the last k memory references at time t
- If the process never accessed the same page twice, we would have $w(k,t) = k$
- But because of reference of locality, $w(k,t)$ stops increasing as k gets large

Working Set – Algorithm

- We can take advantage of the fact that the working set of a process is roughly a fixed size after some warmup period



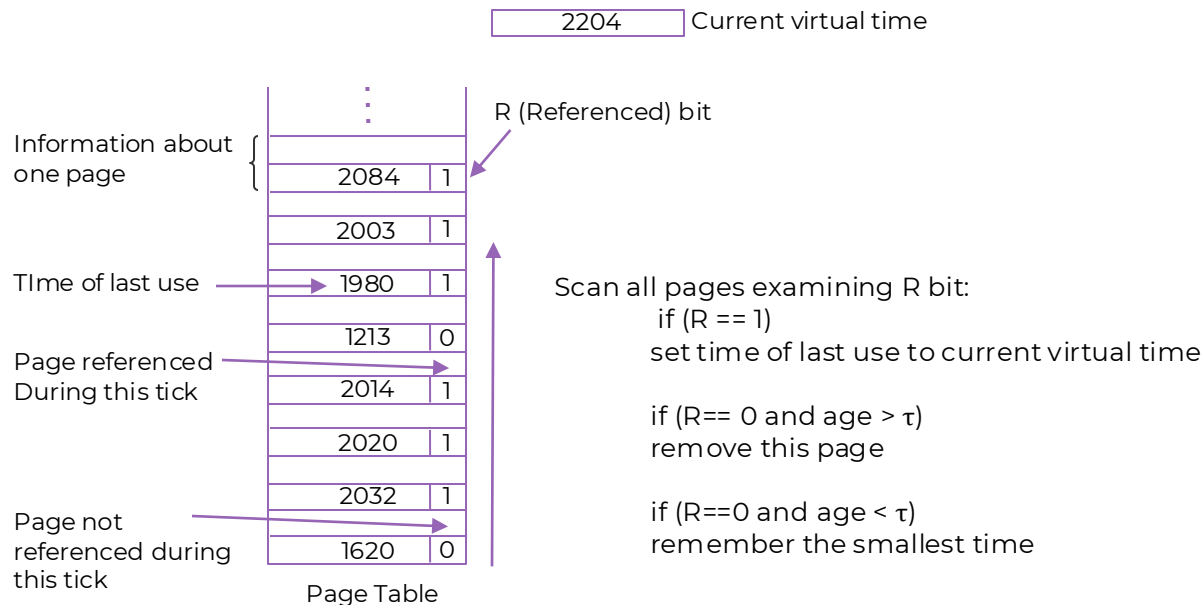
Tracking the Working Set

- For performance, we would like to load the exact working set into the process when it's paged back in
- In theory, we can just fix some value k and then track the pages touched in the last k memory references
- But once again, doing anything on **every** memory reference is very very slow

Approximating the Working Set

- As we saw with LRU, we may be able to make do with an *approximation* of the working set
- For example: pages referenced in the last τ ms
 - Note: only track time when the process is actually executing – its *current virtual time*
- Now we can take advantage of the CPU features that set the "referenced" bit automatically for us

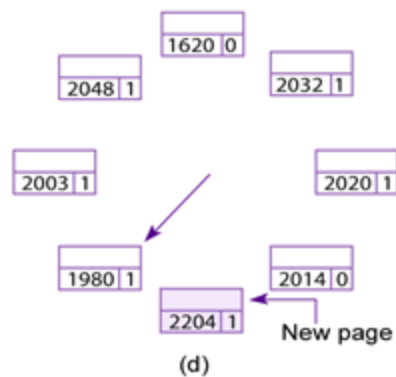
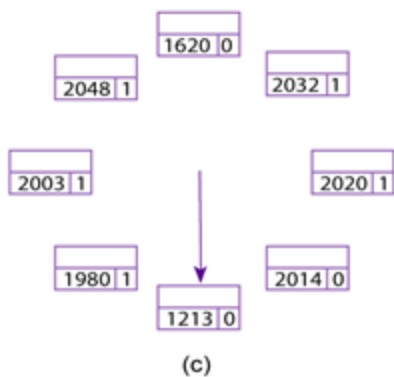
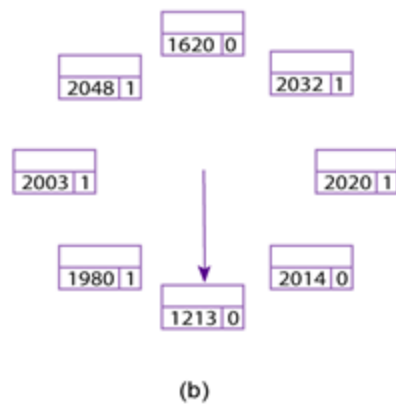
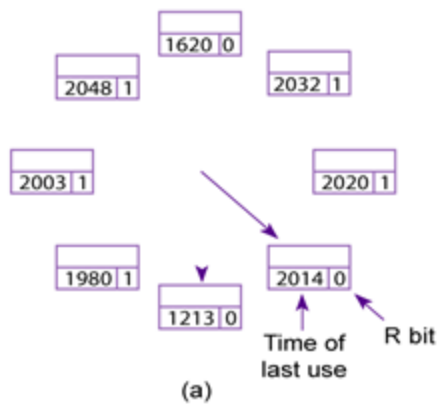
Working Set – Algorithm



WSClock

- Working set algo is cumbersome since entire page table has to be scanned
- Enter WSClock

2204 Current virtual Line



Other Considerations

- Global vs local page replacement
 - When we run our replacement algorithm, does it look just at the pages in the current process, or at pages allocated to all processes?
- Load control
 - Sometimes we can't avoid thrashing
 - One possible solution – put some processes to sleep entirely, and hope that the working set of the remaining ones shrinks

```

    group_info->blocks[0] != group_info->small_
    int i;
    if (groupinfo->blocks[0] != group_info->small_
        for (i = 0; i < group_info->nblocks; i++)
            int i;
            freepage((unsigned long)groupinfo->blo
            for (i = 0; i < group_info->nblocks; i++)
                freepage((unsigned long)groupinfo->blo
            kfree(groupinfo);
        }
        kfree(groupinfo);
    }
    EXPORT_SYMBOL(groupsfree);
    EXPORT_SYMBOL(groupsfree);
    /* export the groupinfo to a user-space array */
    /* export the groupinfo to a user-space array */
    const struct group_info *group_info)
    static int groups_touser(gid_t_user *grouplist,
        const struct group_info *group_info)
    {
        const struct group_info *group_info)

        int i;
        unsigned int count = groupinfo->ngroups;
        int i;

        unsigned int count = groupinfo->ngroups;
        for (i = 0; i < group_info->nblocks; i++) {
            unsigned int cpcount = min(NGROUPSPERBLOCK
            for (i = 0; i < group_info->nblocks; i++) {
                unsigned int len = cpcount * sizeof(*group
                unsigned int cpcount = min(NGROUPSPERBLOCK
                = cpcount * sizeof(*group

```

- Virtual Memory
- Page Replacement Algorithms
- XV6 Implementation

Paging Hardware

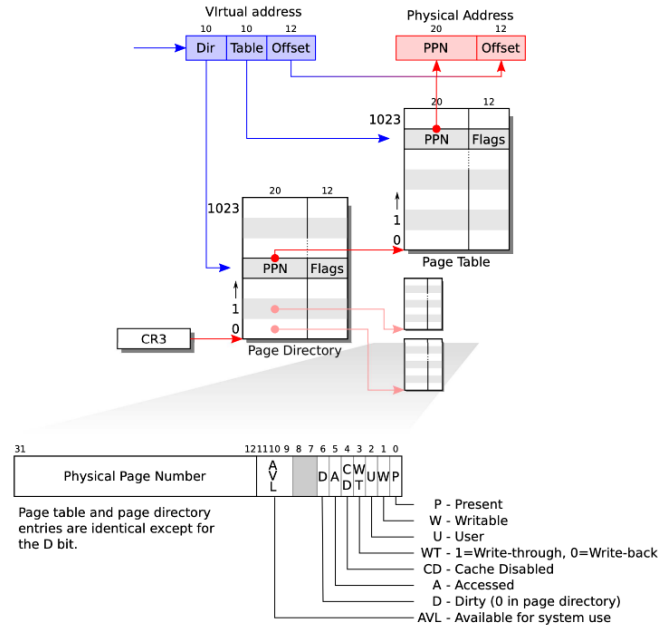


Figure 2-1. x86 page table hardware.

Virtual Address Translation in xv6

- xv6 mostly uses 4KB pages (one page table per process, and **one page table specifically for the scheduler**)
- Early on in **boot**, it uses a var called **entrypgdir**, which creates 4MB mappings

entrypgdir

Page dir entries

main.c

```
// Boot page table used in entry.S and entryother.S.  
// Page directories (and page tables), must start on a page boundary,  
// hence the "__aligned__" attribute.  
// Use PTE_PS in page directory entry to enable 4Mbyte pages.  
__attribute__((__aligned__(PGSIZE)))
```

→ page aligned

```
pde_t entrypgdir[NPDENTRIES] = {  
    // Map VA's [0, 4MB) to PA's [0, 4MB)  
    [0] = (0) | PTE_P | PTE_W | PTE_PS,  
    // Map VA's [KERNBASE, KERNBASE+4MB) to PA's [0, 4MB)  
    [KERNBASE >> PDXSHIFT] = (0) | PTE_P | PTE_W | PTE_PS,  
};
```

→ Set Page → 4 MB Page

0x80000000 >> 22 = 512

```
entry.S  
# Set page directory  
movl    $(V2P_WO(entrypgdir)), %eax  
movl    %eax, %cr3
```

xv6 Process Page Tables

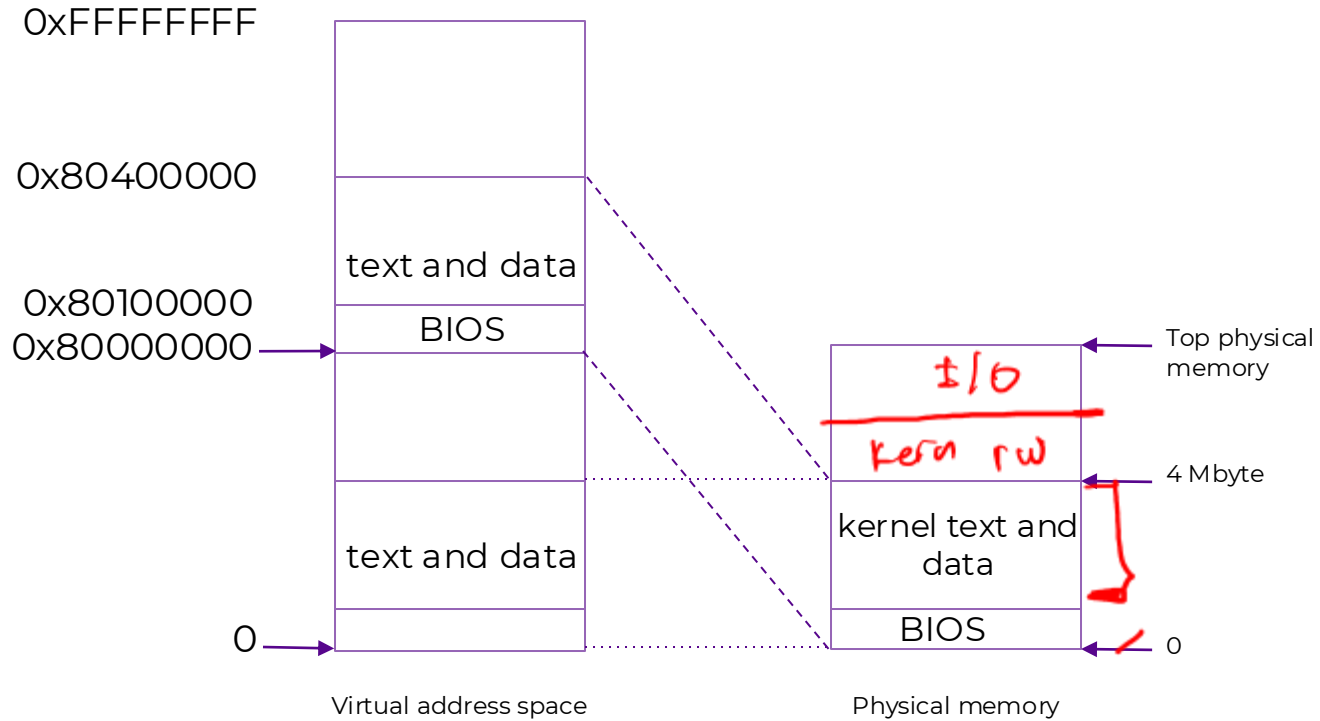
- `entrypgdir` suffices for allowing the the kernel's C code to start running. Once this is done xv6 sets up a more complicated page table
 - I/O space
 - Read-only space for kernel code and r/o data
 - Kernel writeable data & memory

kmap

```
1 // Memory layout
2
3 #define EXTMEM 0x100000 // Start of extended memory
4 #define PHYSTOP 0xE000000 // Top physical memory
5 #define DEVSPACE 0xFE000000 // Other devices are at high addresses
6
7 // Key addresses for address space layout (see kmap in vm.c for layout)
8 #define KERNBASE 0x80000000 // First kernel virtual address
9 #define KERNLINK (KERNBASE+EXTMEM) // Address where kernel is linked
10
```

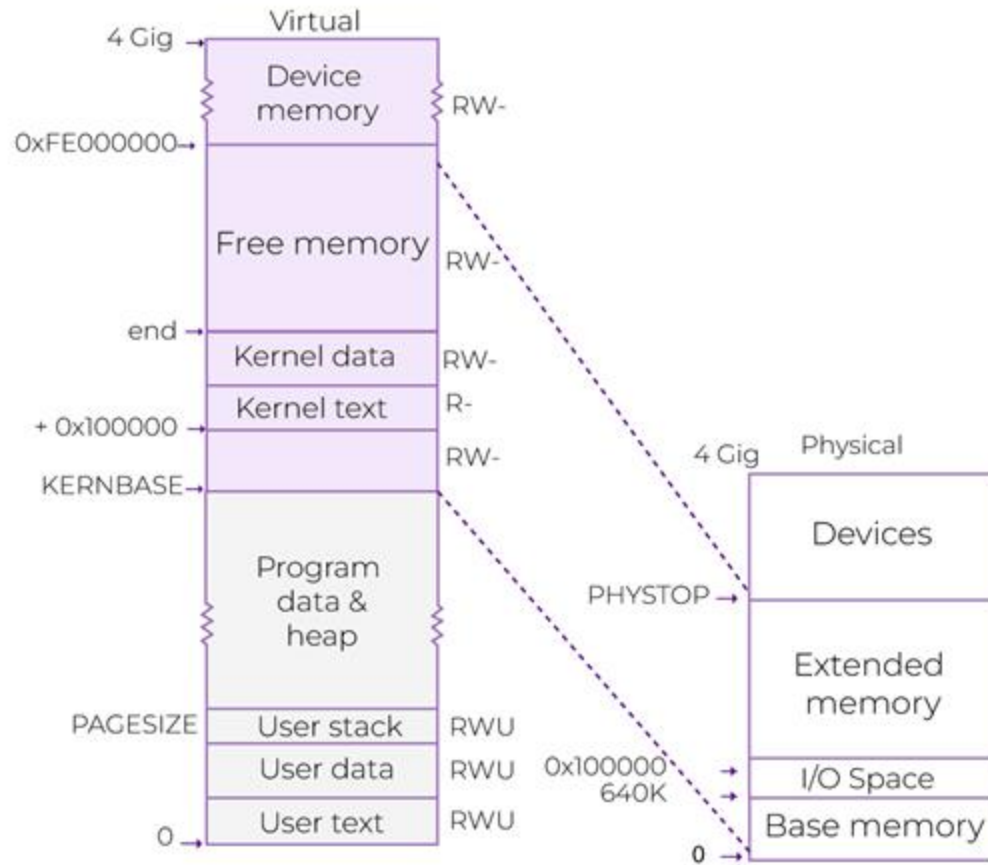
```
// This table defines the kernel's mappings, which are present in
// every process's page table. (vm.c)
```

```
static struct kmap {
    void *virt;
    uint phys_start;
    uint phys_end;
    int perm;
} kmap[] = {
    { (void*)KERNBASE, 0, EXTMEM, PTE_W}, // I/O space
    { (void*)KERNLINK, V2P(KERNLINK), V2P(data), 0}, // kern text+rodata
    { (void*)data, V2P(data), PHYSTOP, PTE_W}, // kern data+memory
    { (void*)DEVSPACE, DEVSPACE, 0, PTE_W}, // more devices
};
```



Implementing the Map

```
// Set up kernel part of a page table.
pde_t*
setupkvm(void)
{
    pde_t *pgdir;
    struct kmap *k;
    if((pgdir = (pde_t*)kalloc()) == 0)
        return 0;
    memset(pgdir, 0, PGSIZE);
    if (p2v(PHYSTOP) > (void*)DEVSPACE)
        panic("PHYSTOP too high");
    for(k = kmap; k < &kmap[NELEM(kmap)]; k++)
        if(mappages(pgdir, k->virt, k->phys_end - k->phys_start,
                    (uint)k->phys_start, k->perm) < 0)
            return 0;
    return pgdir;
}
```



Creating the First Process Address Space

Note: only 4096 bytes given to process to start with

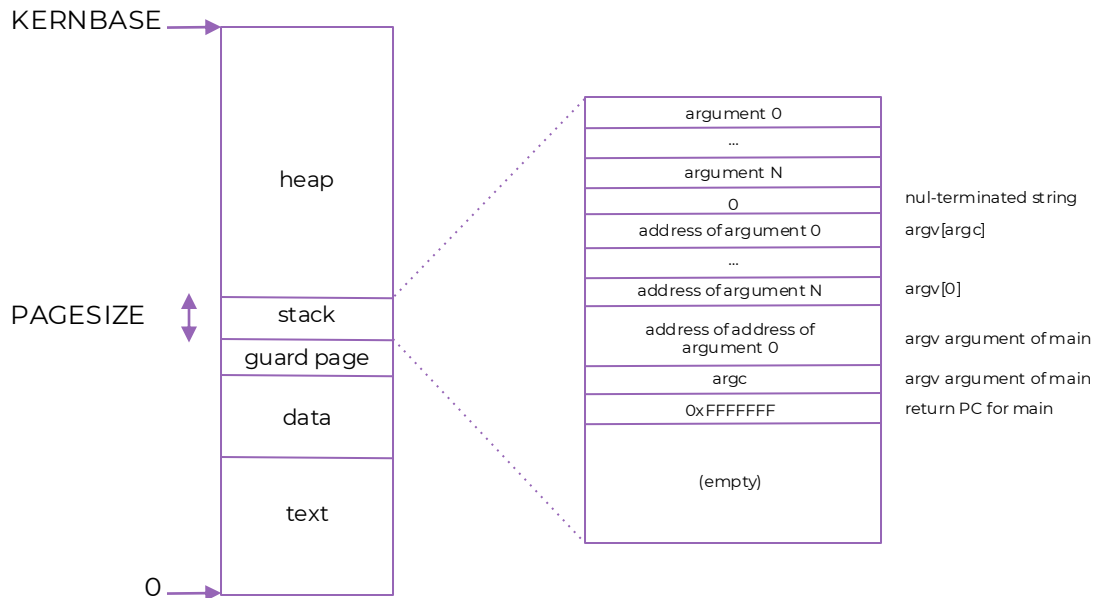
```
// Load the initcode into address 0 of pgdir.  
// sz must be less than a page.  
// #define PGSIZE          4096  
void  
inituvm(pde_t *pgdir, char *init, uint sz)  
{  
    char *mem;  
  
    if(sz >= PGSIZE)  
        panic("inituvm: more than a page");  
    mem = kalloc();  
    memset(mem, 0, PGSIZE);  
    mappages(pgdir, 0, PGSIZE, v2p(mem), PTE_W|PTE_U);  
    memmove(mem, init, sz);  
}
```

Allocate a page of memory for the page directory.

Install the kernel translations

User-Space Layout

Layout set up in `exec()`



Guard Page

```
int
exec(char *path, char **argv)
{
    [...]
    // Allocate two pages at the next page boundary.
    // Make the first inaccessible. Use the second
    // as the user stack.
    sz = PGROUNDUP(sz);
    if((sz = allocvm(pgdir, sz, sz + 2*PGSIZE)) == 0)
        goto bad;
    clearpteu(pgdir, (char*)(sz - 2*PGSIZE));
    sp = sz;
    [...]
}
```

Other Features of xv6 Paging

- Kernel address space is actually shared by all processes
- The upper 2GB maps to the same physical pages.
- Don't flush the TLB. System calls are much faster
- When you switch processes you switch CR3

Paging Tricks

- Having virtual address translation around lets us do lots of interesting things aside from basic process isolation
- We'll go through a few applications people have found that make use of the paging hardware

Paging Tricks: Copy-on-write

- In UNIX, `fork()` creates a complete copy of the memory of another process
- This is really inefficient – takes time to make the copy, wastes memory unless the new process makes changes

Paging Tricks: Copy-on-write

- Instead: set up new process's mappings so they point to the same pages as the old process
- But make the mapping read-only using page permissions.
- Now if the process tries to write to one of its pages,
 - Trap it to the kernel (via a page fault)
 - Make a real copy and then perform the write.

Paging Tricks: Memory-mapped files

- Normal way of reading a file: `read()` some data into a buffer, work on it, `write()` it back out
- Instead we might want to map a file into memory – virtual address $X+N$ would refer to the N th byte in the file
- We can again use paging for this – mark the pages corresponding to the file not present, then when someone tries to read them, read in from disk
- We can write back changes either immediately (by making pages write protected) or when the process exits

Paging Tricks: Lazy Memory Allocation

- Programmers are terrible people and sometimes allocate more memory than they really need
- Example:
 - allocating a large buffer for user input, then only using the very beginning
- We'd like to not give them any more than they are actually using

Paging Tricks: Lazy Memory Allocation

- **Solution:** make the system call that allocates memory essentially **do nothing**
 - But keep track of the fact that the process has allocated that virtual address range
- If the process actually tries to use the memory, we will get a page fault and can then **allocate a page on demand**
- If the pages are never touched, will never allocate

Paging Tricks: Lazy Memory Allocation

- Beware – we can end up handing out more memory than we actually have this way
- This is called *overcommitting*
- Linux actually does this intentionally – malloc() almost never fails
- But if you try to use all that memory, and the system runs out, the dreaded OOM-Killer will be invoked

Paging Tricks: Memory Breakpoints

- When using a debugger, we might want to watch all changes to a particular piece of memory
- E.g., to find out what code sets a global variable
- x86 has hardware support for triggering a debugger when memory is accessed
 - But it's limited to just 4 memory locations
 - No good for monitoring larger data structures, e.g.

Paging Tricks: Memory Breakpoints

- With a bit of help from the OS, we can do better
- Any time we set a memory breakpoint, mark the **entire page** non-present (or read-only for memory write breakpoints)
- Any access will trap into the OS, which can notify our debugger; when the debugger continues, the OS can finish handling the fault